

Cómo funciona el agente guía

El requisito del enunciado es un agente embebido que **monitorea la aplicación y enseña a usarla**. Esto explica cómo está construido, por qué así, y qué contestar si te lo preguntan.

Código en `src/main/java/bo/forja/backend/agente/`.

1. Lo primero que hay que poder decir

Es un agente híbrido con las reglas al mando: primero un sistema experto, y el modelo de lenguaje sólo donde las reglas no llegan. El orden es la decisión.

El argumento: para saber que *una clase marcada como interfaz no puede tener atributos* no hace falta inferir nada, hace falta **saber la regla**. Un modelo de lenguaje lo diría a veces y a veces diría otra cosa. Un sistema de reglas lo dice siempre, se puede auditar y se puede probar.

Y tiene una consecuencia comprobable, que es el argumento fuerte: **las preguntas del catálogo dan siempre la misma respuesta**. Eso se verifica en una prueba. Con un LLM al frente no se podría.

El modelo entra **sólo cuando el catálogo no engancha** — es decir, exactamente donde antes el agente decía «esa no la sé contestar». Es el único lugar donde no puede empeorar nada: lo peor que puede pasar es volver a esa misma respuesta. Y no contesta de lo que el modelo sepa: se le pasa **la misma base de conocimiento escrita como contexto**, con la orden de no salir de ahí. No conoce FORJA; sin ese contexto la inventaría.

En el proyecto conviven los dos paradigmas de IA, cada uno donde sirve:

Paradigma	Dónde	Para qué
Simbólico (reglas)	El agente guía: 24 reglas y 17 respuestas escritas	Razonar sobre el modelo y enseñar la herramienta
Generativo (Gemma)	El pedido de diagrama, el respaldo del dictado, y el agente cuando sus reglas no llegan	Entender y redactar lenguaje natural

Si te preguntan «¿dónde está la IA?», la respuesta no es una sola: son dos, y están separadas a propósito.

Y dos cosas que NO son IA, que conviene no llamar así porque no resisten la repregunta: el **generador de Spring Boot** es un traductor determinista con reglas de mapeo —por eso el código compila siempre y es reproducible—, y el **reconocimiento de voz** (voz→texto) lo hace el navegador o Android, no FORJA. Lo que sí es de la herramienta es texto→comando.

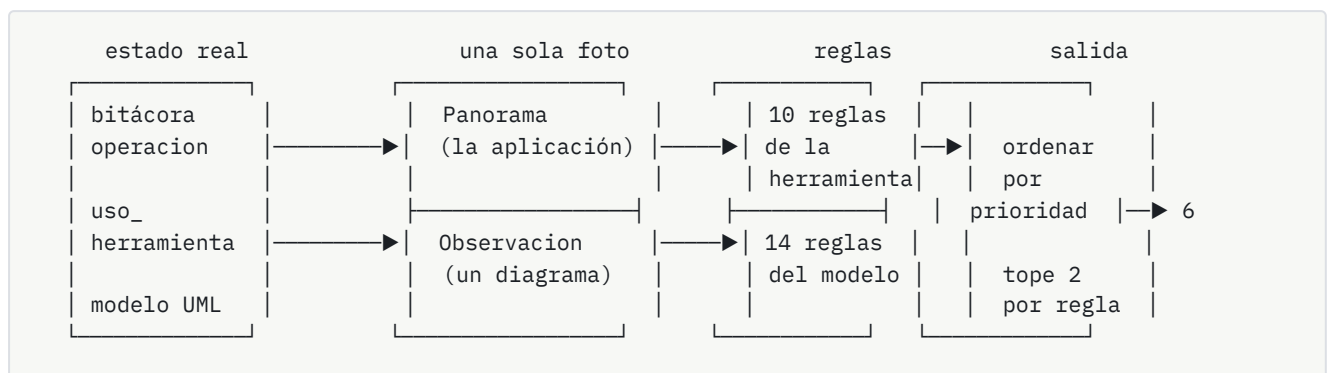
2. Hace tres cosas distintas

	Qué es	Endpoint
Consejos	Qué conviene hacer ahora, mirando el estado real	GET /api/guia
Recorrido	Dónde estás parado dentro de la herramienta completa	va en la misma respuesta
Preguntas	Le preguntás algo y responde: catálogo primero, modelo después	POST /api/guia/pregunta

Los temas que sabe responder se ofrecen en `GET /api/guia/temas`, y hay un `GET /api/diagramas/{id}/agente` para los consejos de un diagrama puntual.

El agente no cambia nada. Solo mira y explica.

3. Cómo produce un consejo



Paso 1 — se arma una sola foto del estado. `Observacion` (un diagrama) y `Panorama` (la aplicación entera).

Existen por dos razones. La técnica: con veinticuatro reglas evaluándose en cada consulta, si cada una hiciera sus propias consultas el agente costaría más que el resto de la aplicación. La importante: **las reglas quedan comprobables sin Postgres** — una observación se construye a mano en una prueba y la base de conocimiento entera se revisa en milisegundos.

Paso 2 — cada regla mira la foto y concluye. Una regla no conoce a las demás, no guarda estado y no consulta nada. Por eso el conjunto se lee como lo que es: una lista de condiciones con su consecuencia, y se puede agregar o quitar una sin tocar el resto. Es la forma clásica de un sistema experto.

Paso 3 — se ordenan y se recortan. Por prioridad, **máximo 6 consejos y máximo 2 por regla.**

Se devuelven pocos a propósito: un agente que muestra veinte avisos a la vez es una lista de pendientes que nadie lee. Lo que puede romper el código generado aparece antes que una convención de nombres.

Los consejos descartados por la persona se filtran y no vuelven.

Un consejo tiene tres campos, no uno

Campo	Responde
queNote	Qué vi
porQueImporta	Por qué te tiene que importar
comoSeHace	Qué hacer al respecto

Y una categoría: `DESCUBRIMIENTO` (algo de la herramienta que no probaste), `MODELO` (algo incompleto que va a afectar lo que se genere) o `DISEÑO` (una mejora que el modelo ya permite ver).

4. Las dos familias de reglas

Son dos y no una sola más grande por un motivo concreto: **alguien que todavía no creó ningún proyecto no tiene diagrama que observar, y es justo quien más necesita que le expliquen por dónde se empieza.**

Reglas de la herramienta — 10 (`BaseDeLaHerramienta.java`)

Enseñan la aplicación. Miran el `Panorama` .

`sin-proyectos` · `sin-diagramas` · `hoja-en-blanco` · `nunca-dicto` · `nunca-uso-la-pizarra` · `nunca-genero` · `nunca-intercambio` · `nunca-invito` · `nunca-pregunto` · `ya-recorrio-todo`

Reglas del modelo — 14 (`BaseDeConocimiento.java`)

Revisan el diagrama. Miran la `Observacion` .

`diagrama-vacio` · `sin-relaciones` · `nunca-dicto` · `listo-para-generar` · `proyecto-de-uno` · `puede-intercambiar` · `clase-sin-atributos` · `clase-aislada` · `sin-clave-propia` · `interfaz-con-atributos` · `raiz-sin-abstracta` · `atributos-repetidos` · `atributo-ya-heredado` · `nombre-en-plural`

Con el lienzo abierto se evalúan **las dos familias juntas** y se ordenan por prioridad. Sin diagrama abierto, solo la primera.

Detalle fino que conviene saber: algunas reglas de la herramienta dicen lo mismo que una del diagrama —«nunca dictaste», por ejemplo—. Cuando el lienzo está abierto la del diagrama lo dice con más precisión, así que **la otra se calla** para no repetir el consejo. Por eso `Panorama` lleva `enUnDiagrama` .

5. El recorrido: 8 pasos con evidencia

Los consejos sueltos enseñan de a una función. El recorrido muestra el camino entero, que es otra cosa: quien recién entra necesita saber **cuánto falta y a dónde lleva todo esto**, no solo cuál es el próximo botón.

1. Crear un proyecto
2. Crear un diagrama

3. Dibujar el modelo
4. Dictarle una frase
5. Leer la foto de una pizarra
6. Generar el backend
7. Intercambiar con Enterprise Architect
8. Invitar a alguien

Ningún paso se marca porque hayas visto una pantalla. Se marcan mirando la evidencia real: la bitácora `operacion` y el registro `uso_herramienta`. El recorrido **no se puede completar haciendo clic en «siguiente»**: hay que haber hecho la cosa.

Esa misma evidencia es la que la evaluación pide para demostrar que cada función existe y se usa. La bitácora guarda el **origen** de cada cambio — `LIENZO`, `VOZ`, `FOTO`, `IMPORTACION`, `AGENTE` —, así que se puede demostrar con datos cuánto del modelo se construyó dictando o fotografiando.

6. Las preguntas

17 entradas escritas a mano en `Preguntas.java`. Cada una tiene su pregunta, sus palabras clave, una prioridad y la respuesta en los tres campos.

Cómo empareja

1. **Normaliza** el texto: recorta a 400 caracteres, saca acentos, pasa a minúsculas y deja solo letras, números, espacios, punto y asterisco. Por eso `¿Cómo exporto a Enterprise Architect?` y `como exporto a enterprise architect` dan exactamente lo mismo.
2. Si es una **repregunta corta** — `¿y eso?`, `no entendí`, `un ejemplo`, `de nuevo` ...— vuelve sobre el último tema en vez de caer en «no la sé contestar».
3. **Puntúa por subcadena**. Cada clave que aparezca en el texto suma, y **una clave de varias palabras vale más**: quien escribe «rombo lleno» está siendo más preciso que quien escribe «rombo». La prioridad solo desempata.
4. Devuelve las **3 mejores**.
5. **Si ninguna enganchó**, y solo entonces, se le pregunta al modelo local con el catálogo entero como contexto. Presupuesto de 25 s.

Las tres salidas posibles, medidas

Pregunta	Camino	Tiempo real
«cómo exporto a Enterprise Architect»	catálogo	0,05 s
«qué pasa si dos editamos la misma clase a la vez»	modelo, con el catálogo de contexto	6,0 s
«hay atajos de teclado»	el modelo dice que no está → vuelve la respuesta escrita	2,0 s

La respuesta del modelo **va etiquetada como tal** (`respuesta-del-modelo`) y avisa que puede equivocarse. Ocultar de dónde salió sería vender como conocimiento de la herramienta algo que no lo es.

El prompt le permite contestar `NO ESTA EN LA DOCUMENTACION` , y lo usa. Ese centinela **nunca se muestra**: se trata como «no contestó» y gana la respuesta escrita, que además ofrece los temas que sí están.

Nunca falla

`POST /api/guia/pregunta` **no valida la entrada, y es a propósito**. Rechazar un texto vacío o larguísimo daría un 400, y un error es justo lo que no puede pasar mientras alguien prueba la herramienta delante de un aula: el agente quedaría mudo en el peor momento. Cualquier texto entra, se recorta si hace falta, y siempre sale una respuesta. Hay una prueba que lo ataca con entradas hostiles.

Limitación honesta, decila vos antes de que te la señalen: el emparejamiento es por subcadena. No entiende que dos frases quieren decir lo mismo. Si alguien pregunta algo razonable con palabras que no están listadas, cae en «no la sé contestar» y ofrece los temas que sí sabe. Se arregla agregando la clave.

Esto pasó de verdad el 20 de septiembre: no contestaba «cómo uso la aplicación» porque la clave era `"como se usa"` , que no es subcadena de esa frase. Está arreglado y hay una prueba que lo fija.

7. Cómo demostrarlo

Que aprende de lo que hacés: entrá con una cuenta nueva y abrí `Guía` . Va a decir que no tenés proyectos. Creá uno, volvé a abrirla: cambió. Dictá una frase: el consejo «nunca dictaste» desaparece y el paso del recorrido se marca.

Que razona sobre el modelo: creá una clase sin atributos → avisa que va a generar una tabla de una sola columna. Marcala como `interface` y ponele un atributo → avisa que una interfaz no puede tenerlos.

Que es determinista donde importa: hacé una pregunta del catálogo dos veces. Palabra por palabra, la misma respuesta, y en milisegundos. Decí que con un modelo de lenguaje al frente eso no se puede garantizar.

Que el modelo recoge lo que las reglas no cubren: preguntale «qué pasa si dos editamos la misma clase a la vez». No está en el catálogo, tarda unos segundos, y contesta bien — porque la respuesta sí está en la documentación que se le pasó de contexto.

Que no se rompe: mandale una pregunta vacía, o basura, o 400 caracteres de texto sin sentido. Siempre contesta.

8. Dónde está cada cosa

Archivo	Qué es
<code>ServicioAgente.java</code>	Orquesta: arma la foto, evalúa, ordena, recorta
<code>Observacion.java</code>	Todo lo que se puede mirar de un diagrama
<code>Panorama.java</code>	Todo lo que se puede mirar de la aplicación
<code>Regla.java</code> / <code>ReglaDeLaHerramienta.java</code>	La interfaz de una regla
<code>BaseDeConocimiento.java</code>	Las 14 reglas del modelo
<code>BaseDeLaHerramienta.java</code>	Las 10 reglas de la herramienta
<code>Recorrido.java</code>	Los 8 pasos y cómo se marcan
<code>Preguntas.java</code>	Las 17 preguntas, el emparejamiento y la base como texto
<code>ia/Respondedor.java</code>	El contrato de «contestar con palabras»
<code>ia/RespondedorOllama.java</code>	La implementación con Gemma, y el prompt que le prohíbe salir del contexto
<code>ia/RespondedorNulo.java</code>	Sin IA: el agente es el sistema experto de siempre
<code>Consejo.java</code>	Qué noté / por qué importa / cómo se hace
<code>ServicioUso.java</code> + <code>AnotadorDeUso.java</code>	El registro de qué funciones probó cada persona

Pruebas en `src/test/java/bo/forja/backend/agente/`: `BaseDeConocimientoTest`, `BaseDeLaHerramientaTest`, `PreguntasTest`, `PreguntasResistenciaTest` (entradas hostiles), `AgenteNoSeCaeTest` y `AgenteHibridoTest`, que fija el ORDEN: una pregunta del catálogo **no puede** llegar al modelo.

9. Los límites, dichos por vos

Conviene adelantarlos en la defensa en vez de que te los encuentren:

- **No aprende.** Las reglas están escritas; no se ajustan solas con el uso.
- **El catálogo empareja por subcadena**, y eso produce falsos positivos: la clave `id` matchea dentro de «servidor». Cuando el catálogo se equivoca así, el modelo ni se entera, porque solo entra si el catálogo no enganchó nada.
- **Sin Ollama, el modelo no está** y el agente vuelve a ser exactamente el sistema experto de antes. La versión desplegada en AWS corre así.
- **La respuesta del modelo no es reproducible.** Por eso está etiquetada y por eso las preguntas que importan están en el catálogo.

- **No razona fuera de lo que se le programó.** No va a descubrir un problema de diseño que no esté escrito como regla.
- **El registro de uso es por persona, no por equipo.** Si tu compañero generó el backend, a vos te sigue diciendo que nunca lo generaste.

Ninguno de esos límites es un accidente: son el precio de que las respuestas sean auditables y siempre las mismas, que es lo que se eligió.